

05_results_comparison

June 8, 2026

1 05 — Comparação de Resultados e Apresentação

TCC PUC — Ciência de Dados e Big Data Evandro Ribeiro Gomes Coelho

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

plt.rcParams['figure.figsize'] = (14, 5)
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3
sns.set_palette('tab10')

PROC = '../data/processed/'
print('OK')
```

OK

```
[2]: # Carrega resultados dos notebooks anteriores
results_baseline = pd.read_csv(PROC + 'results_baseline.csv')
results_advanced = pd.read_csv(PROC + 'results_advanced.csv')
preds_df = pd.read_csv(PROC + 'predictions.csv', parse_dates=['Timestamp'])

# Unifica todos os resultados
all_results = pd.concat([results_baseline, results_advanced], ignore_index=True)
all_results = all_results.sort_values('R²', ascending=False).
    ↪reset_index(drop=True)

print('Resultados carregados:')
display(all_results)
```

Resultados carregados:

Modelo	MAE	RMSE	R ²	MAPE (%)
--------	-----	------	----------------	----------

0	MLP (features expandidas)	3.3644	17.4877	0.99996	0.099
1	Random Forest	37.4732	74.5665	0.99928	0.988
2	XGBoost	95.8274	128.1559	0.99786	2.606
3	Regressão Linear	1083.8078	1499.3586	0.70716	27.487
4	MME (=0.5)	1197.6700	1709.2959	0.61941	29.263
5	MME (=0.3)	1296.0484	1802.9547	0.57656	33.667
6	MME (=0.1)	1519.6861	1996.9895	0.48051	45.349
7	MMS (w=12)	1492.1786	2020.4274	0.46825	41.823
8	MMS (w=6)	1477.4809	2026.9756	0.46479	38.345
9	MMS (w=24)	1639.5833	2150.1403	0.39778	50.720
10	SARIMA	1449.6561	1761.8203	0.36011	29.911
11	MLP (TCC2 original)	2435.5006	2916.4034	-0.10795	75.603

1.1 1. Dashboard de Comparação

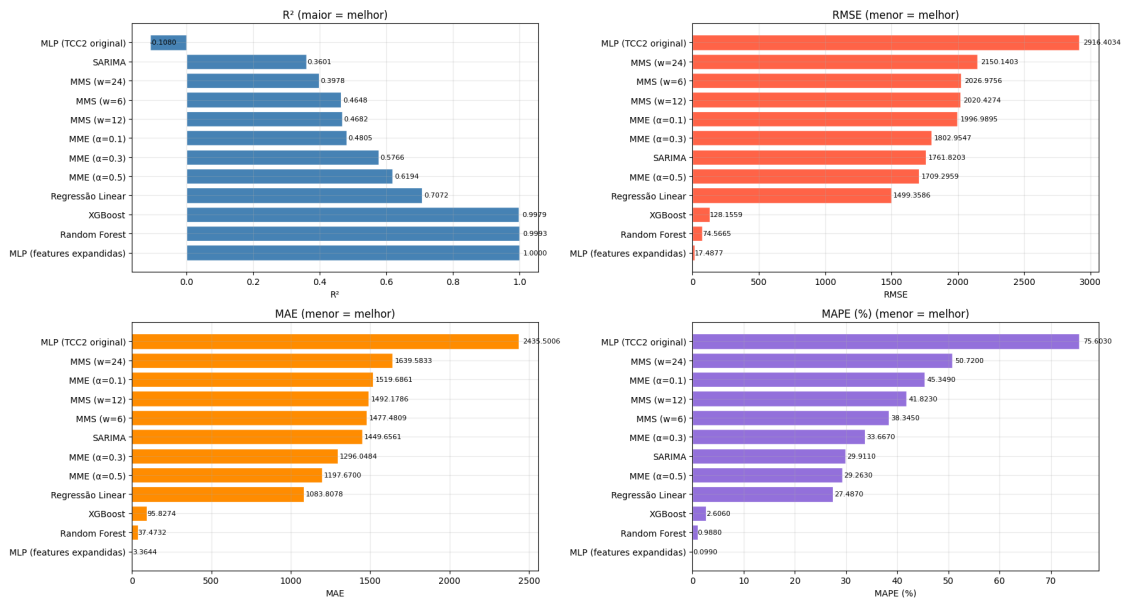
```
[3]: metrics = ['R²', 'RMSE', 'MAE', 'MAPE (%)']
ascending_map = {'R²': False, 'RMSE': True, 'MAE': True, 'MAPE (%)': True}
color_map = {'R²': 'steelblue', 'RMSE': 'tomato', 'MAE': 'darkorange', 'MAPE_
↳ (%)': 'mediumpurple'}

fig, axes = plt.subplots(2, 2, figsize=(18, 10))
axes = axes.ravel()

for i, metric in enumerate(metrics):
    sorted_df = all_results.sort_values(metric, ascending=ascending_map[metric])
    bars = axes[i].barh(sorted_df['Modelo'], sorted_df[metric],
                        color=color_map[metric], edgecolor='white')
    for bar, val in zip(bars, sorted_df[metric]):
        axes[i].text(bar.get_width() * 1.01, bar.get_y() + bar.get_height()/2,
                    f'{val:.4f}', va='center', fontsize=8)
    direction = 'maior = melhor' if not ascending_map[metric] else 'menor =
↳ melhor'
    axes[i].set_title(f'{metric} ({direction})')
    axes[i].set_xlabel(metric)

plt.suptitle('Dashboard de Comparação - Referência vs Avançados', fontsize=14,
↳ y=1.01)
plt.tight_layout()
plt.show()
```

Dashboard de Comparação — Referência vs Avançados



1.2 2. Gráfico Dispersão Real vs. Previsto por Modelo

```
[4]: pred_cols = [c for c in preds_df.columns if c.startswith('pred_')]
n_models = len(pred_cols)
y_real = preds_df['Value'].values

ncols = min(3, n_models)
nrows = (n_models + ncols - 1) // ncols
fig, axes = plt.subplots(nrows, ncols, figsize=(6*ncols, 5*nrows))
if n_models == 1:
    axes = [axes]
else:
    axes = axes.ravel()

label_map = {
    'pred_mlp_tcc2': 'MLP (TCC2)',
    'pred_mlp_new': 'MLP (expandido)',
    'pred_rf': 'Random Forest',
    'pred_xgb': 'XGBoost',
}

for i, col in enumerate(pred_cols):
    y_pred = preds_df[col].values
    r2 = r2_score(y_real, y_pred)
    label = label_map.get(col, col)
```

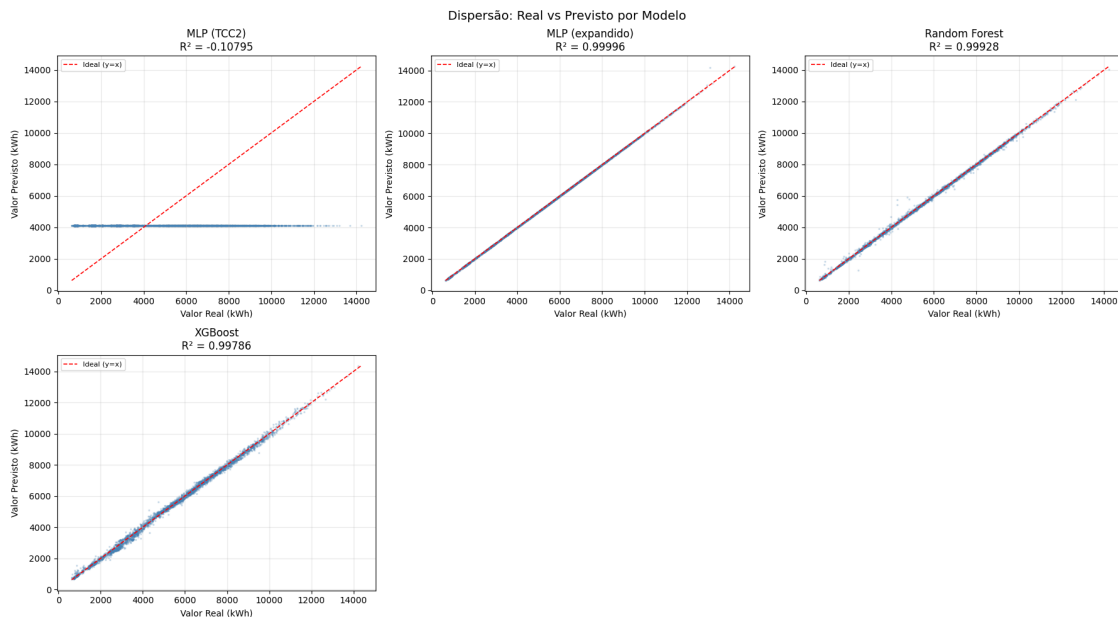
```

axes[i].scatter(y_real, y_pred, alpha=0.2, s=2, color='steelblue')
lim = [min(y_real.min(), y_pred.min()), max(y_real.max(), y_pred.max())]
axes[i].plot(lim, lim, 'r--', linewidth=1.2, label='Ideal (y=x)')
axes[i].set_title(f'{label}\nR² = {r2:.5f}')
axes[i].set_xlabel('Valor Real (kWh)')
axes[i].set_ylabel('Valor Previsto (kWh)')
axes[i].legend(fontsize=8)

# Oculta subplots extras
for j in range(i+1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Dispersão: Real vs Previsto por Modelo', fontsize=14)
plt.tight_layout()
plt.show()

```



1.3 3. Histograma de Resíduos

```

[5]: fig, axes = plt.subplots(nrows, ncols, figsize=(6*ncols, 4*nrows))
if n_models == 1:
    axes = [axes]
else:
    axes = axes.ravel()

for i, col in enumerate(pred_cols):
    residuals = y_real - preds_df[col].values

```

```

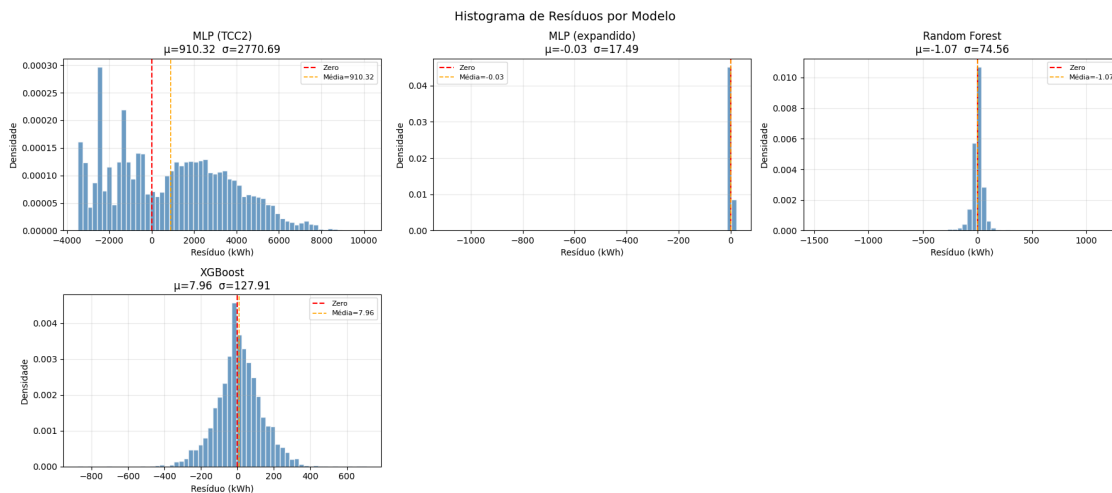
label = label_map.get(col, col)
mean_r = residuals.mean()
std_r = residuals.std()

axes[i].hist(residuals, bins=60, color='steelblue', edgecolor='white',
↪density=True, alpha=0.8)
axes[i].axvline(0, color='red', linestyle='--', linewidth=1.5, label='Zero')
axes[i].axvline(mean_r, color='orange', linestyle='--', linewidth=1.2,
↪label=f'Média={mean_r:.2f}')
axes[i].set_title(f'{label}\nμ={mean_r:.2f} σ={std_r:.2f}')
axes[i].set_xlabel('Resíduo (kWh)')
axes[i].set_ylabel('Densidade')
axes[i].legend(fontsize=8)

for j in range(i+1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Histograma de Resíduos por Modelo', fontsize=14)
plt.tight_layout()
plt.show()

```



1.4 4. Feature Importance (Random Forest + XGBoost)

```

[6]: import os

fi_files = {
    'Random Forest': PROC + 'feature_importance_rf.csv',
    'XGBoost':       PROC + 'feature_importance_xgb.csv',
}

```

```

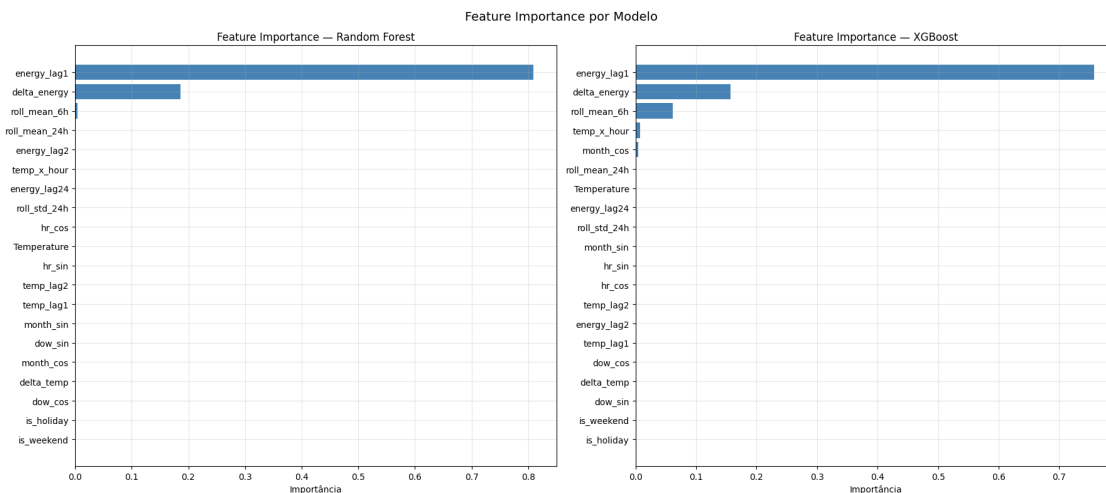
available_fi = {name: path for name, path in fi_files.items() if os.path.
↳exists(path)}

if available_fi:
    fig, axes = plt.subplots(1, len(available_fi),
↳figsize=(9*len(available_fi), 8))
    if len(available_fi) == 1:
        axes = [axes]

    for ax, (model_name, path) in zip(axes, available_fi.items()):
        fi = pd.read_csv(path, index_col=0, header=0)
        fi.columns = ['importance']
        fi = fi.sort_values('importance', ascending=True)
        colors = ['steelblue' if v > fi['importance'].median() else
↳'lightsteelblue'
                    for v in fi['importance']]
        ax.barh(fi.index, fi['importance'], color=colors)
        ax.set_title(f'Feature Importance - {model_name}')
        ax.set_xlabel('Importância')

    plt.suptitle('Feature Importance por Modelo', fontsize=14)
    plt.tight_layout()
    plt.show()
else:
    print('Arquivos de feature importance não encontrados. Execute o notebook
↳04 primeiro.')

```



1.5 5. Tabela Final Consolidada

```
[7]: print('=== RESULTADOS FINAIS CONSOLIDADOS ===')
      print('(Ordenados por R2, decrescente)\n')

      # Destaque: melhor e pior modelo
      best_model = all_results.iloc[0]
      worst_model = all_results.iloc[-1]

      display(all_results.style
               .highlight_max(subset=['R2'], color='#d4edda')
               .highlight_min(subset=['RMSE', 'MAE', 'MAPE (%)'], color='#d4edda')
               .format({'R2': '{:.5f}', 'RMSE': '{:.4f}', 'MAE': '{:.4f}', 'MAPE (%)': '{:.
               ↪3f}'})
               )

      print(f'\nMelhor modelo : {best_model["Modelo"]} (R2= {best_model["R2"]:.5f})')
      print(f'Pior modelo   : {worst_model["Modelo"]} (R2= {worst_model["R2"]:.5f})')
      ganho = ((best_model['R2'] - worst_model['R2']) / abs(worst_model['R2'])) * 100
      print(f'Ganho relativo: {ganho:.1f}%')
```

=== RESULTADOS FINAIS CONSOLIDADOS ===

(Ordenados por R², decrescente)

<pandas.io.formats.style.Styler at 0x117b2e090>

Melhor modelo : MLP (features expandidas) (R²=0.99996)

Pior modelo : MLP (TCC2 original) (R²=-0.10795)

Ganho relativo: 1026.3%

```
[8]: # Salva tabela final de resultados
      all_results.to_csv(PROC + 'results_final.csv', index=False)
      print(f'Tabela final salva: {PROC}results_final.csv')
```

Tabela final salva: ../data/processed/results_final.csv